

Actions de remédiation — supervision de la flotte

Généré le 2026-07-24 12:01 — boucle propose → arbitre → applique : rien ne s'applique sans décision humaine.

1. Findings ouverts (à arbitrer)

PROJET	PRIORITÉ	CIBLE	CONSTAT
VSCode	P3	revue-increment	Increment 6 cloture alors qu'une regression PPTX bloquante est capitalisee, sans passage revue-increment
VSCode	P3	scan_transcripts	Etage-1 diagnostique a l'aveugle : 0 session couverte, state/runs vides -> jamais_utilises est un artefact de donnees vides
VSCode	P2	bmad-catalogue-codex	Duplication .agents/skills/ (Codex) : poids mort structurel, independant du scan
VSCode4	P1	ppt-designer	Contournement du cadre photo des dividers de chapitre jamais re-questionné, malgré l'écart documenté au pattern VSCode3 que le dispositif est censé répliquer

2. Pratiques repérées par la veille (règle + action proposées)

Gestion du contexte outillée : statusline de suivi, /compact cadré, sous-agents d'exploration

Anthropic — Claude Code docs (reduce token usage, context window) + best practices (context is the fundamental constraint) — statut nouveau

Pourquoi : La doc fait de la fenêtre de contexte LA ressource à gérer (statusline de suivi, /clear entre tâches, /compact ciblé, sous-agents pour l'exploration) ; la flotte a la discipline ÉCRITE sur VSCode1/VSCode3 seulement (sections optimisation tokens), rien de mesuré, et rtk (proxy token-optimisé) est déployé sans suivi de son gain réel.

Règle d'analyse proposée : Critère scan (dimension pratiques + rules) : discipline tokens écrite dans le CLAUDE.md/conventions du projet (marqueurs /compact, sous-agent, lecture ciblée) — mesurable 0 token.

Action corrective : Propager la section « optimisation tokens » (modèle VSCode1/VSCode3, adaptée au canal) aux CLAUDE.md de VSCode/VSCode2/VSCode4/VSCode5 ; mesurer le gain rtk (rtk gain --history) à chaque veille et le reporter dans le wiki.

3. Remédiations déjà appliquées (exemples commentés)

DATE	CIBLE	DÉCISION APPLIQUÉE (COMMENTAIRE)
2026-07-23	famille:linter	ACCEPTÉ + APPLIQUÉ (« traite tout ») : ruff configuré (pyproject.toml, baseline F/I/UP/B) sur VSCode2 (102 points mesurés, B008 ignoré car idiome FastAPI) et VSCode5 (0 point après nettoyage). PAS de --fix aveugle : un ruff --fix sur VSCode2 a retiré un ré-export (is_configured) et cassé un import — reverté, tests re-verts (40 passed) ; correction au fil de l'eau. VSCode/VSCode3/VSCode4 hors périmètre (peu de code).
2026-07-23	famille:revue-code	ACCEPTÉ + APPLIQUÉ (« traite tout ») : hook warn_verif_before_commit porté de VSCode1 vers VSCode2, marqueurs adaptés (pytest/pptx-verify/revue-increment au lieu de npm test), câblé en PreToolUse. Rappel non bloquant au commit si du code app/ part sans vérif réelle dans la session. Fail-open vérifié. L'agent reviewer dédié reste optionnel (coût).
2026-07-23	famille:design-review	ACCEPTÉ + APPLIQUÉ (« traite tout ») : skill deck-design-review greffée sur VSCode4, RÉÉCRITE pour son deck OHC réel (15 slides : couverture, sommaire, dividers chapitre teardrop, leviers, personas, architecture, existant, évaluation, mentorat, arbitrages, séquençement, roadmap) et son canal (rendu LibreOffice, générateur generate_deck_ohc.py) — pas un copier-coller de VSCode2. VSCode3 reste à traiter.
2026-07-23	famille:cadrage-produit	ACCEPTÉ + APPLIQUÉ (« traite tout ») : product-brief rédigé pour VSCode5 (docs/product-brief.md — persona = orchestrateur de flotte, why = ne pas re-découvrir/re-corriger le même écart projet par projet, besoins = voir/alerter/comparer/approfondir/corriger/veiller, proposition de valeur = piloter la flotte comme un seul système via la boucle propose→arbitre→applique). VSCode4 (deck RH) reste à cadrer sur demande. Autres projets : fragments suffisants.
2026-07-23	famille:documentation	ACCEPTÉ + APPLIQUÉ (« applique la reco ») : le hub VSCode5, jusque-là sans README ni CLAUDE.md, en est doté — README.md (ce que fait le dispositif, démarrage, skills, structure, boucle d'amélioration) et CLAUDE.md (règles R1-R5 pour agir sur la flotte : lire l'état réel,

commit scopé, adapter au canal, propose→arbitre→applique, vérité du journal + vérifications avant commit). Dimension documentation VSCode5 : moyen → ok. Non traité sur VSCode/VSCo3/VSCo4 (README via bmad-document-project sur demande) — hors périmètre de cet arbitrage.

2026-07-23	famille:tests	ACCEPTÉ + APPLIQUÉ (« applique la reco pratique-test ») : couverture de test outillée sur les 2 projets à forte densité de test, via le playbook evolution-flotte. VSCode2 : pytest-cov ajouté à requirements-dev.txt, commande documentée dans conventions.md, 1ère mesure ~38 % (pptx_deck 56 %, pptx_export 48 %). VSCode1 : c8 en devDep + script npm test:cov + étape CI, 1ère mesure 84,67 % lignes. Aucun seuil imposé (on mesure d'abord). Non traité sur VSCode/VSCo3/VSCo4 (peu de code / pré-code) — hors périmètre assumé.
2026-07-23	scan_transcripts.py	ACCEPTÉ + APPLIQUÉ (« traite tout ») : le scan compte désormais les skills invoquées en slash-command (<command-name> filtré sur les skills installées), le hint revue-increment est conditionnel à la présence réelle de la skill, et le fix slug (caractères non alphanumériques → tiret) est propagé. Recomptage complet de ce projet fait (agent-orchestrator ×3, agent-supervisor ×1 mesurés). Les 3 édits sont propagés aux 5 autres copies de la flotte par édits ciblés vérifiés (py_compile vert partout).
2026-07-23	log_run.py	ACCEPTÉ + APPLIQUÉ (« traite tout ») : mode --solde <prefixe-ts> <resultat> "note" ajouté (requalification tracée d'un run en-attente-validation — la validation utilisateur devient un événement de première classe du journal, testée sur les chemins nominal et erreurs). Propagé aux 5 autres copies (copie directe sur les identiques, insertion ciblée sur les divergées). Le bandeau du wiki affiche la commande.
2026-07-23	playbooks	ACCEPTÉ + APPLIQUÉ (« traite tout ») : playbook evolution-flotte créé (cadrage sur l'état réel → modification scopée → vérifications → commit limité au périmètre → wiki → journal), enregistré au catalogue et dans la table des playbooks de l'orchestrateur, statut éprouvé (capitalisé des 4 runs flotte du 2026-07-23).
2026-07-24	famille:dispositif-partage	ACCEPTÉ + APPLIQUÉ (P1 de l'analyse craft) : la dette risque_technique CRITIQUE (audit VSCode5 — 6 copies divergentes de scan_transcripts.py/log_run.py maintenues à la main) est résorbée par un canon unique + synchronisation, remplaçant la propagation manuelle par édits ciblés du 2026-07-23 (l'approche qui avait laissé diverger les copies). Canon dans .claude/dispositif/canon/, propagé

par sync_dispositif.py (en-tête « généré — ne pas éditer localement », modes --check/--projet). Le cadrage a révélé que la dette n'était pas uniforme : version canonique de fait partagée par 3 projets + 2 améliorations éparpillées jamais réunies — détection des skills consommées par lecture (ex-VSCode1) et arbitrage restreint par catégorie (ex-VSCode3). Le canon = UNION des deux : chaque projet GAGNE ce qui lui manquait, aucun ne perd. Commits scopés (2 fichiers/cible + dispositif/ au hub) et push main sur les 6 remotes ; scans relancés 6/6 sans échec. Suivi : re-lancer audit-technique VScode5 pour sortir la dimension de « critique ».

2026-07-24	canon-p1-regression	ACCEPTÉ + APPLIQUÉ (arbitrage utilisateur « garder A, fixer le test ») : l'enrichissement A du canon (non_invocation_skills — détection des skills consommées par lecture) a fait échouer 1 test de la suite VSCode2 (un exemple de test supposait qu'un skill sans dossier scripts/ était « jamais utilisé »). Décision : conserver l'amélioration A (comportement voulu) et corriger le test — l'exemple devient priority-matrix (skill sans scripts/ authentiquement non invoquée). Suite complète VSCode2 re-vérifiée VERTE (330 passed) avant tout commit aval. Illustre la valeur du canon partagé : la régression est apparue et a été traitée UNE fois, pas 6.
2026-07-24	risque-technique:VSCode1-scores	ACCEPTÉ + APPLIQUÉ (P2 volet VSCode1) : le cœur de calcul (agrégation de scores / moyenne / écart-type), sans aucun test unitaire (finding audit VSCode1 risque_technique), est extrait dans scores.js et couvert par test-scores.js — npm test vert. R1 a évité de refaire la couverture (c8 déjà en place, 84,67 % — arbitrage famille:tests du 2026-07-23) : seul le trou réel (test du calcul) est comblé. Reste optionnel : brancher le test du chemin export dans npm test.
2026-07-24	famille:integration-continue	ACCEPTÉ + APPLIQUÉ (P3) : CI GitHub Actions ajoutée à VSCode2 — seul projet Python avec tests + couverture + ruff mais sans CI (finding pratique-dev, capability DORA « intégration continue »). Workflow sur push main + PR : Python 3.12, install requirements-dev, ruff en NON bloquant (baseline assumée, « mesure d'abord »), pytest --cov=app comme gate. Modèle : la CI JS de VSCode1. Le volet linter de P3 (ruff sur VSCode/VSCode3/VSCode4) reste REFUSÉ/hors périmètre — déjà arbitré le 2026-07-23 (peu de code).
2026-07-24	famille:pratiques-providers	ACCEPTÉ + APPLIQUÉ (« lance les chantiers 1 à 4 ») : les 4 pratiques du volet 2 de veille-agentic passent en adopte. (1) Hook de vérif pré-commit propagé à VSCode/VSCode3/VSCode4, adapté au canal de chacun (COMOP smoke-test / générateur pytest), commits scopés poussés ; en chemin, la même régression canon-P1 que VSCode2

détectée sur le test VSCode3 et corrigée selon l'arbitrage existant (exemple deck-design-library, 29 verts). (2) Règle scan « CLAUDE.md ≤ 150 lignes » (fonction pure + 7 tests) + élagage VSCode1 158→149 et VSCode3 285→107 (changelog historique du deck retiré, contraintes durables conservées). (3) Étape revue-fraiche (contexte frais, sonnet) ancrée dans evolution-flotte et export-ppt-verifie (JSON validés). (4) Hub aligné garde-fous en couches : 6 deny rules (secrets locaux + secrets des projets CIBLES qu'il lit), dimension sécurité  → .

2026-07-24	famille:dispositif-package	ACCEPTÉ + APPLIQUÉ (demande « package de déploiement agentic ») : .claude/dispositif/package/deploy_nouveau_projet.py — bootstrap complet d'un nouveau projet SANS copies au repos (leçon P1) : manifeste de 21 sources vivantes (canon du hub, hooks hub/VSCode3, kit d'export VSCode2, tests VSCode3) matérialisées à la demande + settings.json câblé (4 hooks + deny rules) + squelette CLAUDE.md (< 150 l) + arbitrages.json vide. Modes --check (sources vivantes) et --force. Vérifié par déploiement d'essai réel : 25 fichiers, tout compile, JSON valides, 26/29 tests verts pré-BMAD (3 rouges attendus, documentés dans la checklist avec l'install BMAD). Leçon capturée : basetemp court obligatoire (MAX_PATH Windows).
2026-07-24	wiki-site-actions	ACCEPTÉ + APPLIQUÉ (demandes « boutons déclencheurs + onglets + exports PDF ») : le wiki devient un site web — 5 onglets thématiques (Pilotage/Projets/Pratiques & risques/Veille/Actions & exports, hash persistant, nav sticky) ; onglet Actions avec déclencheurs 0-token (re-scan, sync --check, package --check, régénération PDF) et LLM via claude -p (diagnostic, audit par projet, veille, remédiation par finding — gouvernance propose→arbitre→applique préservée dans les prompts) ; serveur scripts/serve_wiki.py (127.0.0.1 UNIQUEMENT — leçon audit VSCode 0.0.0.0 —, allowlist stricte, suivi de jobs) testé bout en bout ; 2 exports PDF téléchargeables (analyse détaillée par projet avec findings localisés = exemples et synthèses = commentaires ; actions de remédiation avec propositions de veille et arbitrages appliqués commentés), régénérés via Edge headless, vérifiés visuellement.